INTERNAL DOCUMENTATION

Service Material Tracker
How the App Works

A plain-language explanation of what this application does, how a faulty part flows through it from start to finish, and what every file in the codebase is responsible for.

1. What is this application?

When a field engineer replaces a faulty part (a Power Card, Control Card, or a motor/pump) at a farmer's site, that faulty part doesn't just disappear — it has to travel through a repair/inspection pipeline before it either gets fixed and put back into the spare-parts stock, sent to an external repair vendor (OEM), or scrapped.

This app is the system that tracks that journey. Every person who touches the part — the engineer, the warehouse staff, the service desk, the electronics repair technician — logs into their own page, sees only the items relevant to them, and records what they did. An admin can see everything, across every stage, at once.

It's a **Django** web application (Django is a Python framework for building websites) backed by a **MySQL** database.

2. The journey of one faulty part (the “material flow”)

This is the heart of the whole system. Not every part goes through every step — there are a couple of shortcuts — but this is the full path:

1. ENGINEER
   Replaces the faulty part on-site, logs it, and dispatches it
   (physically ships it) to a Warehouse.
       │
       ▼
2. (optional) REGIONAL WAREHOUSE
   Some parts land at a regional warehouse first. Staff there either
   deal with it directly (Tested OK / Repaired / send to OEM) or
   forward it on to the Main Warehouse.
       │
       ▼
3. MAIN WAREHOUSE — "Receive"
   Checks the part in: records the date received and whether the
   physical serial number matches the paperwork.
       │
       ├──► Power/Control Card whose serial starts with "P" or a
       │   motor/pump: skips straight to step 6 (OEM/Scrap decision) —
       │   these never go through Service Desk or Electronics.
       │
       ▼
4. SERVICE DESK
   Runs a mandatory inspection checklist (visual check, MTR test, cold
   test, wiring) and hands the part on to Electronics.
       │
       ▼
5. ELECTRONICS
   Runs a detailed diagnostic (what failed, which component, etc.) and
   reaches a conclusion: Repaired OK, Tested OK, or Scrapped.
       │
       ▼
6. MAIN WAREHOUSE — final disposition
   Based on the conclusion, the warehouse either:
     • Adds it back to service stock (repaired/tested-OK parts), or
     • Dispatches it to an external OEM for repair, or
     • Scraps it.

Every single step above is written to a permanent history log, so nobody ever loses track of where a part is or where it's been — that's what powers the Admin's "Material Lookup" search (see the User Manual).

3. Who uses the app (the roles)

Every login belongs to exactly one **role**, and that role decides which page you land on and what you're allowed to do. The roles are:

ROLE	WHAT THEY DO
Engineer	Logs and dispatches faulty parts from the field.
Regional Warehouse	An optional first stop; can resolve simple items directly or forward them on.
Warehouse (Main Warehouse)	Receives parts, decides where they go next, and closes the loop (add to stock / OEM / scrap).
Service Desk	Runs the mandatory inspection checklist before Electronics.
Electronics	Diagnoses the fault and reaches a repaired/tested/scrapped conclusion.
Admin	Sees and manages everything — users, all materials, reports, and search.

4. The Admin Dashboard

Because no single operational page shows the *whole* picture, there's a separate Admin area with four sections:

- **Dashboard Overview** — every material in the system, in one table, with a tab for every stage of the pipeline above (Pending, Warehouse, With Service Desk, With Electronics, Dispatched to OEM, Scrapped, Added to Stock, etc.), filterable by part type, engineer, and date.
- **User Management** — add new logins, change a password or role, and activate/deactivate accounts.
- **Reports & Analytics** — two charts: how many materials are at each stage, and which components/faults come up most often (pulled from Electronics' diagnostic notes).
- **Material Lookup** — search by Service ID, machine number, or faulty serial number, and see that part's *entire* history, from the moment the engineer dispatched it to wherever it is now.

> See the User Manual for step-by-step screenshots of all of this.

5. How the project is organized (file by file)

Django projects are split into a **project** (the overall settings/config) and one or more **apps** (the actual features). This project has one app, called `tracker`, which contains everything described above.

Top-level project files

FILE/FOLDER	WHAT IT'S FOR
`manage.py`	The command-line entry point — used to start the server, apply database changes, etc. Nobody edits this.
`requirements.txt`	The list of external Python libraries the app depends on (Django itself, the MySQL driver, etc.).
`config/settings.py`	The app's master configuration: which database to use, which features are turned on, security settings.
`config/settings_local.py`	The *real* database passwords/hostnames for this specific machine. Deliberately kept out of source control — every developer/server has their own copy.
`config/urls.py`	The very top-level web address map (currently just hands everything off to the `tracker` app).
`config/wsgi.py`	A technical file that lets a production web server (like gunicorn) talk to the Django app.
`Dockerfile`, `docker-compose.yml`, `docker/`	Packaging so the whole app (and its database) can be run inside Docker containers — used for deployment.

Inside the `tracker` app

FILE	PLAIN-LANGUAGE ROLE
`models.py`	**The data.** Defines every "thing" the app remembers — a User, a Dispatch, a DispatchItem (one physical faulty part) and all the fields it can have (status, which engineer, which reservation numbers, the electronics diagnosis, etc.), plus the append-only history log (`DispatchItemStatusEvent`) that records every stage a part has passed through.
`views.py`	**The logic.** For every page, this is the code that decides what to show and what happens when someone clicks a button (e.g. "Receive", "Dispatch to OEM", "Add User"). Each role's home page and each action button has its own section here.
`forms.py`	**The input rules.** Defines what fields appear in each pop-up form and what counts as valid input.
`urls.py`	**The address book.** Maps each web address (like `/main-warehouse/`) to the piece of logic in `views.py` that handles it.
`backends.py`	**The bouncer.** Checks a login attempt's email/password against the right *role*.
`services.py`	Shared helper logic used by multiple pages — e.g. "get every item currently with the Service Desk."
`db_router.py`	Sends one particular type of read-only data to a *different* database server than everything else the app writes.
`admin.py`	Hooks into Django's own built-in admin site (not the custom Admin Dashboard — not used day-to-day).
`apps.py` / `__init__.py`	Boilerplate Django needs. No real content.
`tests.py`	Placeholder for automated tests (currently empty).
`migrations/`	A running history of every change ever made to the database's structure.

`templates/` — the actual pages people see

Each `.html` file here is one page. They're plain HTML with placeholders that `views.py` fills in with real data.

- `base.html` — the shared header/footer for every operational page.
- `login.html` — the login screen.
- `engineer_home.html` , `warehouse_home.html` , `main_warehouse_home.html` , `service_desk_home.html` , `electronics_home.html` — one home page per role.
- `admin_base.html` — the shared sidebar/top-bar shell for the Admin Dashboard.
- `admin_dashboard.html` , `admin_users.html` , `admin_reports.html` , `admin_lookup.html` — the four Admin sections.

`static/tracker/` — styling and interactivity

- `style.css` — the shared look for every operational page.
- `admin.css` — the look of the Admin Dashboard specifically.

- One `.js` file per page family — handles pop-up forms, filtering/sorting, expanding row details, and the searchable dropdown in the Electronics form.
- `ecozen-logo.png`, `photovoltaic-2138992_1280.jpg` — images used in the header and login background.

6. Where the data actually lives

- **MySQL, database `fault_tracker`** — everything this app itself creates: users, dispatches, and the full status history of every part. This is the database the app can write to.
- **A second, external MySQL database (`clappia`)** — a *read-only* connection to another team's system, holding the original service-ticket data. The app reads from it but never writes to it — that's why `db_router.py` exists.

7. In one sentence

A faulty part gets logged by an engineer, physically travels through a chain of warehouses and inspection/repair teams (each recording their own step), and ends up back in stock, at an outside repair shop, or scrapped — and an admin can see and search that entire journey for every part, at any time.
